Proceedings of the ASME 2026
International Design Engineering Technical Conferences and
Computers and Information in Engineering Conference
IDETC-CIE2026
August 23-26, 2026, Houston, TX

DETC2026-194240

SEMAA: AN AGENTIC AI APPROACH TO SYSTEM MODELING

Ardalan Aryashad, Rufus Marcussen, Yan Jin*

IMPACT Laboratory
Dept. of Aerospace & Mechanical Engineering
University of Southern California
Los Angeles, California 90089
aryashad@usc.edu, rmarcuss@usc.edu, yjin@usc.edu
(*corresponding author)

ABSTRACT

Agentic AI represents a transformative paradigm in artificial intelligence, offering the potential to move beyond simple chat interfaces toward autonomous systems that can be effectively integrated into complex, real-world professional environments. While large language models (LLMs) have demonstrated remarkable proficiency in general reasoning and standard software development, a significant gap remains in their application within the engineering domain, specifically in Model-Based Systems Engineering (MBSE). Unlike general programming, MBSE relies on highly specialized graphical modeling tools and rigorous standards like SysML, which present a unique challenge for AI integration. The complexity of these tools, combined with a scarcity of domain-specific training data compared to mainstream programming languages, has slowed down the development of effective AI assistants for systems architects.

This paper introduces the SEMAA framework, designed to bridge this gap by establishing a seamless, context-aware system between conversational AI and professional MBSE environments. We developed a custom plugin for an MBSE modeler, featuring an interpreter that treats diagrams as a design language using PlantUML syntax. This allows the AI to programmatically create and modify diagrams within the modeling software, ensuring that the generated artifacts are not merely static images but functional model entities.

To further enhance the system's technical capability, we implemented a multi-agent orchestration strategy. By moving beyond single-agent systems, we distributed responsibilities among specialized agents, including an orchestrator for task decomposition and a specialist for SysML syntax verification. This architecture was evaluated against single-agent baselines using our curated SEMAA benchmark dataset. Our results demonstrate that the multi-agent configuration significantly outperforms single-agent models in terms of relevancy, structural complexity, and technical correctness. This work provides a scalable proof-of-concept for the future of agentic AI systems in engineering.

Keywords: Agentic AI, Model-Based System Engineering, Multi-agent Orchestration, Automated Diagram Generation

1 INTRODUCTION

As mechanical engineering systems evolve toward unprecedented complexity, such as integrating autonomous navigation, hybrid systems, and data-centric operations, traditional design methodologies are increasingly strained. To manage this complexity, Model-Based Systems Engineering (MBSE) has emerged as an industry trend, utilizing formalized modeling to represent system requirements, behaviors, and architectures. However, the adoption of MBSE remains limited in many engineering sectors due to the steep learning curve associated with standards and the high cognitive overhead required to maintain consistency within professional modeling tools.

The recent rise of Large Language Models (LLMs) and agentic AI offers a promising solution to these challenges. While general-purpose AI can assist with text generation and basic coding, it lacks the technical rigor and direct integration required for professional systems engineering. Most existing AI tools are disconnected from the source of truth. Or in other words, the

normal chatbots cannot use the graphical MBSE tools where engineering models actually live. Furthermore, generalist models often struggle with the specialized syntax of standard language, the long-term retention of context necessary for iterative design, and the compliance checks against the requirement documents.

To bridge this gap, we present SEMAA, a Systems Engineering Modeling Assistant with AI. SEMAA is not merely a conversational assistant but a context-aware bridge that embeds an agentic assistant directly into the engineering workflow. By utilizing a custom plugin and a bidirectional interpreter, SEMAA enables engineers to interact with their models through natural language, translating high-level intent into precise, executable modeling actions.

Central to our approach is the use of PlantUML [1] as an intermediate representation. This strategy allows the AI to operate on a text-based structure and decouples it from the modeling software's API, enabling the system to generate and modify diagrams with high reliability. Furthermore, we address the limitations of single-agent systems by implementing a multi-agent orchestration framework. This design delegates responsibilities such as requirement extraction and syntax verification to specialized agents, ensuring that architectural decisions remain grounded in technical standards. Through this integration, SEMAA transforms MBSE from a time-consuming, manual process into a collaborative, AI-augmented design experience.

The contribution of this work can be summarized as follows. We propose a framework for an AI-assisted systems engineering platform that combines a conversational agentic core with a professional MBSE modeling environment. We developed a functional proof-of-concept plugin for Visual Paradigm that utilizes a bidirectional interpreter to translate between natural language, PlantUML code, and formal SysML model elements. We demonstrate the technical advantages of using a specialized multi-agent system over monolithic models, showing improvements in model correctness and complexity while maintaining low operational latency. We provide an evaluation of modern LLMs in both single-agent and orchestrated configurations, offering insights into the relationships among model capacity, orchestration logic, and engineering output quality.

In the rest of this paper, we first review related work in Section 2, then introduce our proposed SEMAA framework and provide implementation details in Sections 3 and 4. The results and discussion of our experiment are in Section 5. Section 6 concludes the paper and points to future research directions.

2 RELATED WORK

2.1 Systems Engineering and MBSE

Systems Engineering (SE) has long been recognized as a disciplined, holistic methodology for designing, analyzing, and managing complex engineered systems. Traditionally, SE relied heavily on document-centric workflows that are prone to inconsistency, misinterpretation, and redundant work. In contrast, Model-Based Systems Engineering (MBSE) represents a fundamental paradigm shift, in which formal models replace disparate documents as the primary medium of engineering knowledge and communication. These models, governed by standardized languages and toolchains, serve as the authoritative single source of truth, improving real-time traceability, reducing manual synchronization errors, and enabling automated verification, simulation, and impact analysis throughout the system lifecycle. MBSE's value becomes especially pronounced in domains with extreme complexity where systems integrate mechanical, electrical, software, and environmental interactions. By centralizing system structure, behavior, and requirements within interconnected models, MBSE supports earlier detection of design flaws and facilitates cross-disciplinary collaboration. This capability directly addresses key challenges in large-scale engineered systems: maintaining coherence across engineering domains, managing cascading impacts of change, and sustaining stakeholder alignment [2].

2.2 SysML Modeling Standards and Tools

The Systems Modeling Language (SysML) is the modeling standard for MBSE, providing a structured visual and textual language to represent system architecture, behavior, interfaces, constraints, and requirements in complex engineering models. Originally derived as an extension of UML, SysML has been standardized and widely adopted across disciplines that require complex representation of engineered systems [3]. Its strength lies in supporting multiple diagram types that capture system structure and behavior simultaneously, enabling engineers to reason about requirements, designs, verification, and validation in an integrated modeling environment.

SysML modeling tools function as the practical interface between engineering teams and the abstract language standard. These environments provide visual editors, model repositories, simulation and validation capabilities, and increasingly, extensibility through plugins and automation frameworks. Such extensibility is critical for SEMAA because it enables the development of custom integrations of AI-based assistant tools that enhance traditional modeling workflows.

Cameo Systems Modeler [4] is one of the most widely adopted commercial SysML modeling environments. It supports comprehensive SysML modeling across the system lifecycle and is often the choice for large enterprise MBSE programs due to its robust feature set, extensive diagram support, and ecosystem maturity. Visual Paradigm [5] provides another SysML-compatible modeling environment, notable for its accessibility and intuitive user interface. It supports all major SysML diagram types, facilitating rapid visual modeling and early system specification exploration. While it appeals to smaller teams and academic users, its plugin and extension capabilities establish a foundation for integrating intelligent assistance layers or domain-specific augmentations. An important open-source alternative is SysON [6], designed from the ground up to support SysML v2 and minimize entry barriers for systems engineers. Its web-based architecture, graphical and textual views, and open APIs make it especially suitable as a platform for extensibility,

community contributions, and integration with custom workflows. In conclusion, SysML modeling tools serve as the execution platforms for the MBSE methodology, and their ongoing evolution toward SysML v2 positions them as ideal hosts for agentic systems like SEMAA. These tools' support for plugins and custom extensions enables the integration of intelligent assistants that can improve modeling quality, lower the barrier to adoption, and transform traditional SE workflows.

2.3 Agentic AI Technologies

A central idea in agentic AI is to treat the language model not as a standalone problem solver, but as a controller that decides when and how to use external tools. Instead of relying solely on text reasoning, the model invokes APIs, executes software, retrieves data, and incorporates outputs into its next step. This model of an orchestrator view is particularly aligned with MBSE, where engineering work is already performed through structured tools such as modelers, simulators, and requirement databases. Another important insight in this line of work is that performance depends heavily on the quality of the interface between the agent and its working environment. Structured interaction protocols, well-defined tool schemas, and disciplined input-output formatting considerably improve reliability and task completion in complex environments. These ideas are developed in Toolformer [7], which trains language models to decide when to call APIs and how to integrate the results; SWE-agent [8], which emphasizes principled agent–computer interfaces for reliable performance in software engineering settings; Z-Space [9], which focuses on large-scale tool orchestration across extensive tool catalogs.

As tasks grow more complex, a single agent becomes insufficient. A common strategy is to decompose work into multiple cooperating agents, each assigned a specific role (e.g., planner, executor, verifier), coordinated through structured message passing. This approach mirrors systems engineering practice, where different specialists contribute to a shared objective. These systems typically provide reusable coordination patterns that enable them to do multi-step problem solving with intermediate checkpoints and recover from partial failures. AutoGen [10] formulates LLM applications as configurable multi-agent conversations; AgentScope [11] provides modular abstractions and infrastructure for robust multi-agent workflows; and Magentic-One [12] adopts a generalist orchestrator coordinating specialist agents for multi-step, tool-using tasks.

Software development has become a primary testbed for multi-agent design because code execution and testing provide fast, objective feedback. As a result, many concrete workflow patterns have emerged that are transferable to engineering modeling contexts. A common pattern is collaborative generation and refinement: one agent generates an artifact, another critiques it, and additional agents debug or optimize it [13,14]. Other approaches introduce hierarchical organization to scale problem solving [15], or adaptive role assignment to improve robustness and correctness [16]. In addition, Self-reflection and structured critique loops are frequently used to detect and correct errors before final output [17]. Although these works target software code, their structure is directly related to MBSE workflows: model synthesis corresponds to code generation, consistency checking parallels debugging, and simulation feedback resembles test execution.

In many engineering contexts, multiple tools can perform similar analyses but differ in fidelity, computational cost, and required expertise. An effective agent should therefore decide not only which tool to use, but also when to escalate from a lightweight check to a high-fidelity analysis. Hierarchical organization and disagreement-based signaling are two strategies for dynamic tool recruitment. MapAgent [18] demonstrates hierarchical agents for geospatial reasoning, while DART [19] uses disagreement among agents to trigger the recruitment of specialized tools.

Modern MBSE increasingly relies on data-rich evidence, including simulation results, test logs, and operational datasets. Several multi-agent systems focus specifically on iterative data analysis workflows, automation of analytical pipelines, and governance of data quality. Tapilot-Crossing [20] benchmarks interactive, iterative data analysis behaviors; AutoML-Agent [21] automates end-to-end machine learning pipelines using multi-agent coordination; LAMBDA [22] frames end-to-end analysis as a data agent problem. For MBSE, these works highlight how agents can manage iterative analysis workflows, enforce data governance principles, and integrate heterogeneous evidence sources into structured decision-making processes.

3 SEMAA ARCHITECTURE

Based on the above critical review of model-based systems engineering practice and the progress of LLMs and agentic technologies, we propose a SEMAA architecture, shown in **Figure 1** and described below.

3.1 An Agentic Approach

The core of the SEMAA architecture is a shift from generative artificial intelligence to an agentic AI paradigm. While traditional Large Language Models (LLMs) operate primarily as chatbots that provide textual responses to users' prompts, SEMAA is designed as an agent capable of planning and goal-oriented execution. In this framework, the LLM does not *act* only as a passive information retrieval model, but it can interact with tools and information to help engineers achieve their goals. The agentic system can interact with external environments, such as modeling software and engineering databases, to fulfill engineers' objectives. As a result, SEMAA moves beyond static question–answering or code-generation workflows and instead provides continuous, context-aware assistance aligned with the iterative nature of model-based systems engineering (MBSE).

3.2 Agent Autonomy

To achieve this level of autonomy, SEMAA leverages the reasoning and planning capabilities of recent LLMs [23]. When presented with a high-level engineering task, for instance, "Create a use-case diagram for an Underwater Autonomous Vehicle (UAV)," the system does not attempt to generate a final design in a single step. Instead, it can reason and decompose the

request into a series of logical sub-tasks. This iterative process is the core of the SEMAA workflow: the assistant generates a preliminary model, observes the output for syntax or logic errors, and waits for the engineer's response to further refine the design until it meets expectations. This iterative "plan-act-reflect" cycle ensures that the resulting models are not just syntactically correct but architecturally sound.

3.3 Operating Engineering Tools

Central to this agentic capability is tool-augmented intelligence, in which the LLM is embedded within a broader system that enables interaction with external tools via function calls or similar mechanisms. In SEMAA, as shown in **Figure 1**, the agent is equipped with a toolbox of modeling-related functions, such as creating or modifying diagrams, querying the document dataset, and performing calculations, which serve as an operational bridge between the output of traditional LLMs and the actual workflow of engineers. This capability effectively transforms SEMAA from a text generator into an assistant for model-based systems engineering tasks.

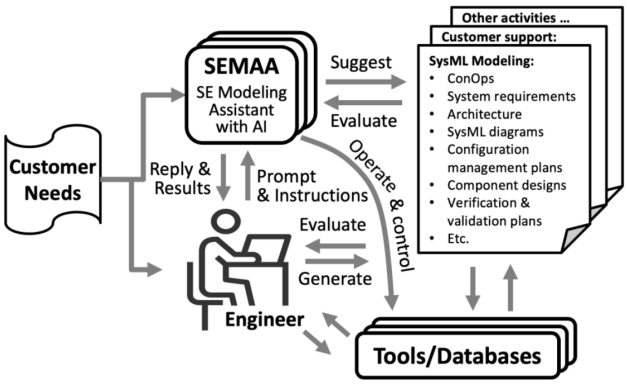

Figure 1: The Proposed SEMAA Framework

3.4 An Operational Workflow

The operational workflow of the SEMAA framework facilitates a dynamic collaboration between the human engineer and the agentic system, fundamentally altering the traditional MBSE lifecycle. As illustrated in **Figure 2**, the process begins with a natural-language User Command (Step A), such as a request to "Create a use case diagram for an Autonomous Underwater Vehicle (AUV)." Unlike traditional MBSE, which requires engineers to manually navigate complex modeling interfaces to instantiate every element, SEMAA triggers Agent Processing (Step B) to autonomously decompose high-level intent into actionable subtasks, such as identifying key actors and logical relationships.

Crucially, the framework ensures design integrity through a Context Memory Check (Step C), which retrieves prior design iterations and relevant maritime safety standards, such as technical rules. This gathered context is synthesized during LLM Integration (Step D), ensuring that the reasoning remains technically grounded. The resulting output is then seamlessly transitioned to the modeling environment via Tool Integration (Step E), where the agentic logic is translated into formal SysML diagrams within the modeler software. This process introduces a Feedback & Validation loop (Step F). In the current MBSE workflow, errors in logic or missing architectural components are often discovered in the design review phase; however, SEMAA provides immediate validation warnings and Result Presentation (Step G) with detailed suggestions for improvement. This culminates in a stage of Iterative Refinement (Step H), where the engineer can refine the model through natural dialogue, repeating the cycle until the architecture meets all technical requirements.

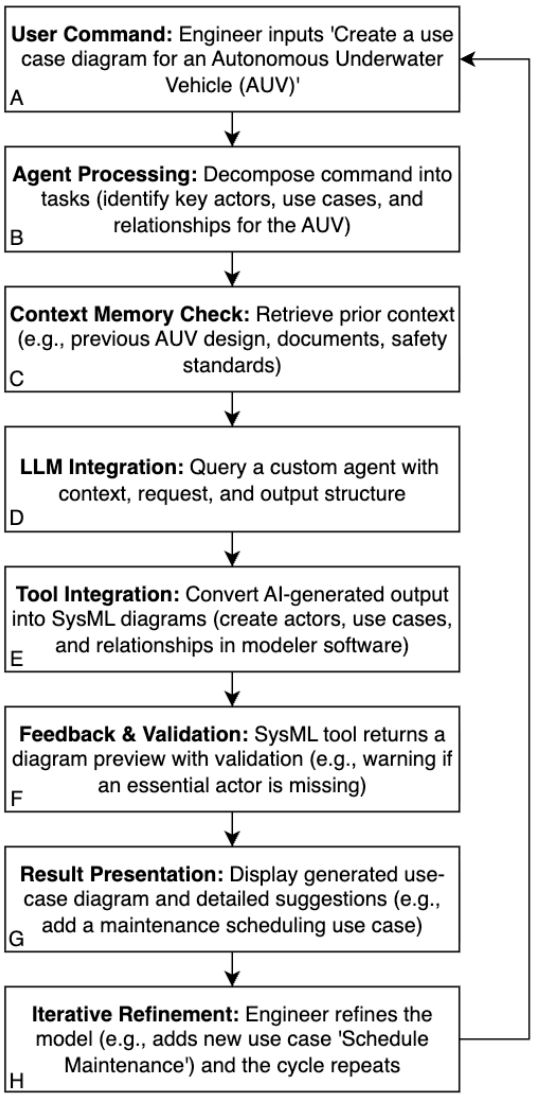

Figure 2: A Workflow Using SEMAA

The advantages of this workflow over traditional MBSE methods are significant. Traditional modeling is often defined by a steep learning curve and the manual effort required to maintain consistency across various diagrams. By contrast, SEMAA's automated orchestration allows for higher architectural Complexity and Correctness without the labor-intensive manual

overhead. Furthermore, the system manages these complex tasks with reasonable latency, ensuring that the AI provides real-time assistance rather than becoming a bottleneck in the design process.

3.5 Agent Orchestration

The final component of the SEMAA architecture is the transition from a monolithic, single-agent system to a collaborative multi-agent orchestration framework. While single-agent systems have shown remarkable capabilities in general tasks, they face significant limitations when applied to the multi-phase workflow of Model-Based Systems Engineering (MBSE). A single agent tasked with reading a design or system modeling document, reasoning through complex documents and requirements, generating multiple SysML diagrams, and performing verification often suffers from context window bloat. As the conversation history grows, models frequently experience "context rot" or the "Lost in the Middle" phenomenon, in which the AI's ability to retrieve and use information degrades significantly when the relevant data is in the middle of a long prompt [24]. This leads to decreased accuracy and the "forgetting" of critical design constraints established earlier in the engineering process.

To mitigate these cognitive bottlenecks, SEMAA utilizes a specialization design inspired by the MetaGPT paradigm [25]. By splitting responsibilities among a team of agents, the system reduces the individual cognitive load on any single model. In our architecture, the roles are divided as follows:

- The Orchestrator Agent: Acts as the coordination layer, responsible for high-level task decomposition, planning the modeling sequence, and routing information among specialist agents.
- The Requirements Agent: Retrieves and structures domain information from user input, project context, and relevant technical documents so that modeling decisions remain grounded in requirements.
- The PlantUML/SysML Generation Agent: Converts the structured requirements and orchestration plan into PlantUML based SysML diagram definitions that can be interpreted by the MBSE tool.
- The Evaluation Agent: Reviews the generated model representation for syntactic validity, structural consistency, and alignment with the user's intent before the result is passed to the modeling environment.

This modularity allows the deployment of heterogeneous models: smaller, faster models can handle repetitive syntactic checks. In contrast, larger, higher-reasoning models are reserved for complex architectural decision-making. Research has demonstrated that such role-playing and collaborative frameworks consistently outperform single-agent methods in software engineering and complex reasoning benchmarks.

A key technical advantage of this orchestration is the implementation of a scoped context management strategy. Rather than passing the entire project history to every agent, which would lead to context rot and high costs, each specialized agent receives only the specific context snippet relevant to its immediate task. For instance, the SysML Expert is provided only with the current diagram definition and the specific architectural constraints to be validated. This approach prevents the context window from being overwhelmed by irrelevant data, thereby improving the precision of agent outputs. While this multi-step interaction introduces a slight latency trade-off due to inter-agent communication, it significantly reduces token costs. It dramatically improves the system's reliability, making it suitable for professional-grade engineering workflows

4 SEMAA REALIZATION

4.1 System Design

The framework, as shown in **Figure 3**, is divided into three primary subsystems: the User, the SEMAA core, and the MBSE Modeler.

User: The User represents the engineer responsible for designing or refining a system by providing detailed instructions and verifying the AI's work through an iterative process.

SEMAA Core: At the center of the design is the SEMAA core, which contains four main parts: the chat interface, the agent, the context manager, and a pair of components known as the splitter and interpreter. The chat interface serves as the main window for communication, where the user writes prompts and reviews responses, while the agent acts as the system's brain, following a specific system prompt that defines its scope, behavior, and the tools it can use. To maintain continuity, the context manager tracks the conversation history and summarizes key details so the agent can refer back to them during long design sessions.

Splitter and Interpreter: Act as the bridge to the MBSE Modeler, converting the agent's text into visual diagrams and translating existing models back into a format the agent can understand. This layout ensures that the MBSE Modeler keeps its full functionality, allowing it to render diagrams while giving the user the freedom to manually modify the design at any time within a professional software environment.

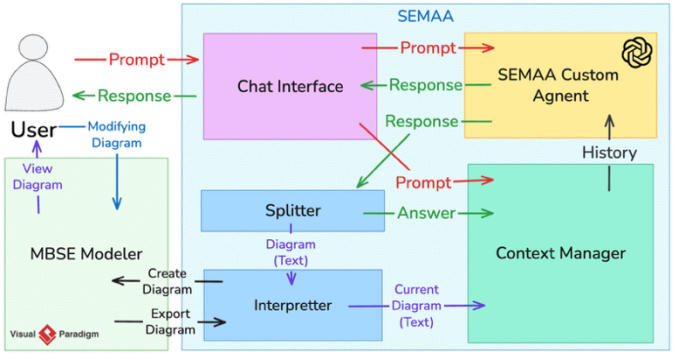

Figure 3: System Design and Implementation

This design is motivated by the need to support a truly iterative engineering workflow, where the design is not a one-time generation but a continuous process of refinement and correction. By decoupling the interface from the modeling tool,

we create a seamless environment where the user can move back and forth between conceptual discussion in the chat and formal implementation in the MBSE software. While general-purpose Large Language Models (LLMs) and Vision-Language Models (VLMs) possess impressive broad knowledge, they lack the specific constraints required for systems engineering; therefore, we utilize custom system prompts to narrow the agent's focus and define its responsibilities, which significantly improves the precision of its technical output compared to "out-of-the-box" models.

Context and History Managers: Crucial to the success of this interaction is the context and history manager, which serves as the system's memory. In complex engineering projects, maintaining consistency is a major challenge; a robust history manager ensures that every design choice is grounded in previous requirements, preventing the context rot that degrades the quality of long-form design sessions. Furthermore, although modern VLMs can interpret visual information, we intentionally avoid passing diagrams solely as images because connections and nested details are easily lost or misinterpreted by visual AI. Instead, the splitter and interpreter work together to treat the diagram as structured code, ensuring that the API calls to the MBSE modeler are perfectly accurate for rendering. This bidirectional capability is vital, as it allows the interpreter to translate manual changes made by the engineer in the modeling tool back into code for the agent. This ensures that the AI remains fully aware of the current state of the design, creating a synchronized environment where both the human and the agent can work on the same model without conflict.

Unlike standard AI tools that often produce static text or disconnected code snippets, the expected output of this system is a functional, editable SysML model integrated directly into a professional modeling environment. When an engineer interacts with the system to define a complex system, such as an autonomous navigation system, the framework can produce structured diagrams in which every element is clearly defined and created within the MBSE modeler. This ensures that any diagram created by the AI is not merely a drawing but a formal model entity that can be further refined, analyzed, or reused by the engineer. Furthermore, the system is designed to generate a natural-language rationale for its modeling choices, providing the traceability needed to justify architectural decisions during the design review process.

Multiple Agents: A central design question in the development of agentic systems is whether to utilize a monolithic single-agent or a multi-agent orchestration. While a single-agent approach - relying on one model for reasoning, history management, and diagram generation - may appear more straightforward, it frequently encounters significant performance bottlenecks in complex engineering scenarios. As the design session progresses, the accumulation of conversation history causes the model's performance to degrade, lose track of critical constraints or fail to follow specific syntactic rules. In contrast, the multi-agent orchestration adopted in this framework leverages task specialization to maintain high design quality. By dividing responsibilities among a team of agents, such as an orchestrator for high-level planning and a specialized expert for technical modeling, the system effectively reduces the cognitive load on any individual model. This modularity enables the framework to use a scoped context strategy, in which each agent receives only the information necessary for its sub-task. This prevents the reasoning engine from being overwhelmed by irrelevant data, thereby improving the accuracy of generating complex diagram structures. Furthermore, this collaborative design enables an autonomous verification process: one agent can act as a reviewer for another's work, identifying potential logical errors or syntactic inconsistencies before the final design is updated. This shift from a single generalist to a coordinated team of specialists is essential for handling the rigorous requirements and high complexity of engineering systems.

4.2 Implementation

The implementation of the SEMAA framework aims to bridge the gap between high-level engineering tasks and standard LLM inference. This section details the selection of the underlying systems and technologies that were used in the implementation.

Large Language Model: The selection of the Large Language Model (LLM) was based on a comprehensive survey of current state-of-the-art models, including the OpenAI GPT series, Anthropic's Claude, and Google's Gemini. While several models demonstrate high competency in general reasoning, the OpenAI suite was selected due to its versatile ecosystem of specialized models that align with the specific requirements of SEMAA framework. For instance, the framework utilizes high-capacity reasoning models to extract and synthesize engineering knowledge from complex documentation, while small models can be employed for rapid response during real-time modeling sessions. Furthermore, the GPT series demonstrated superior reliability in code generation and instruction-following, which is essential for producing syntactically correct modeling code without the errors common in general-purpose models.

SysML Modeling Tool: Parallel to the LLM selection, the choice of a Model-Based Systems Engineering (MBSE) environment was conducted through a comparative evaluation of different options such as Visual Paradigm, Cameo Systems Modeler, and SysON. The framework ultimately adopted Visual Paradigm as its primary modeling host, cited for its cost-effectiveness, comprehensive support for SysML diagram types, and, most critically, its robust OpenAPI capabilities. The availability of a Java-based open API was a decisive factor, as it provided the necessary tools to develop a tightly coupled plugin that could programmatically manipulate the diagram repository inside the software. This extensibility ensured that the proposed SEMAA assistant could function as a native component of the engineering environment rather than an external, disconnected tool.

Java-Based Plugin using PlantUML: The core of the implementation is the SEMAA Plugin, a Java-based extension that functions as a context-aware bridge between the engineer and the AI agent. The architectural philosophy of the plugin is centered on the use of PlantUML as an intermediate

representation. By treating PlantUML as code, the system decouples the proprietary API of the modeling tool from the agent tool. In this design, the AI is not required to navigate the complex internal objects of the MBSE tool; instead, it reasons in a structured, text-based modeling language, which the plugin subsequently translates into a formal diagram. The lifecycle of this integration is managed by a centralized plugin controller that registers with the application's manager and injects a dedicated "SEMAA AI Assistant" interface into the Visual Paradigm primary ribbon shown in **Figure 4**.

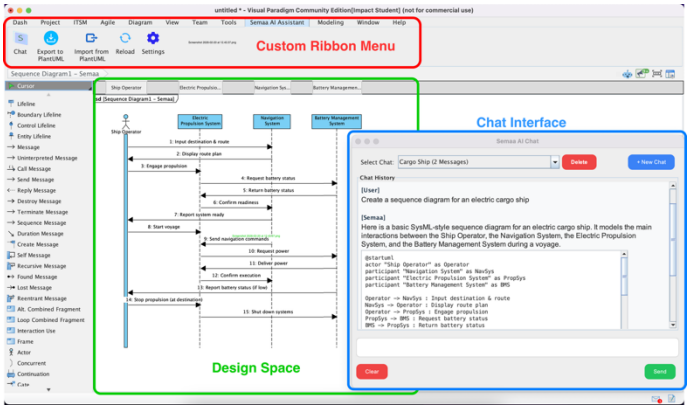

Figure 4: SEMAA AI Assistant inside Visual Paradigm

The components of the SEMAA discussed in the previous section are technically realized through a set of specialized Java modules within the plugin. The Chat Interface is implemented as a native dialog box, providing a seamless interaction window within the VP. Behind this interface, the Agent uses a dedicated chat service to manage communication with the selected OpenAI models. To maintain architectural consistency, the Context Manager handles the conversation history and the active state of the current design session. Before a query is sent to the agent, the context manager automatically captures the current state of the active diagram, ensuring that the AI sees what the engineer is working on without requiring a manual description.

The bidirectional translation between natural language and model objects is managed by the Splitter and Interpreter modules, which are implemented as distinct technical pipelines. The Splitter serves as the extraction tool for obtaining structured PlantUML code from the agent response. In addition, the Interpreter serves as the import pipeline; it parses the AI-generated code and uses API calls to create and lay out a new diagram on the VP. A novel contribution of this system is its specialized handling of Requirement Diagrams. Because standard PlantUML syntax lacks support for this diagram, we developed a custom syntax that enables the agent to reason about and generate it. This shows that the SEMAA agent can also design and create a custom-formatted diagram.

Context Awareness: The system's context awareness is operationalized through a context manager that facilitates bi-directional information flow. Before processing any user request, the service automatically performs Context Extraction by serializing the active diagram's state into a textual representation. This ensures that the agent is fully aware of the current architectural state without requiring the user to manually describe existing components. This interaction is possible because the interpreter can also convert the current diagram into PlantUML code. This process converts visual diagrams into structured code by traversing the model elements within the current diagram. It identifies semantic data such as blocks, ports, and associations, and generates a syntactically correct PlantUML equivalent.

By integrating these pipelines directly into the MBSE tool, the SEMAA plugin demonstrates a viable proof-of-concept for a generative modeling environment in which the AI serves as a collaborative partner in the engineering process.

4.3 Orchestration Design and Multi-Agent System

The multi-agent version of the SEMAA framework is designed to decompose complex engineering tasks into modular, manageable sub-processes. Rather than relying on a single, monolithic model to handle every stage of the design process, the system employs a separation-of-responsibilities strategy. This approach ensures that specialized agents handle distinct cognitive tasks, such as strategic planning, information retrieval, and formal syntax generation, thereby reducing the reasoning burden on any individual component and improving the overall correctness of the output.

At the heart of this design is the Orchestrator Agent, which serves as the system's mind. When a user provides a request, the Orchestrator does not immediately attempt to solve the problem. Instead, it performs a high-level analysis to determine the optimal execution path. It evaluates the task's complexity and identifies which specialized agents are required and in what order. By producing a structured execution schema, the Orchestrator minimizes the risk of model drift, in which the AI loses track of the original goal. This design mirrors the Plan-and-Execute pattern common in advanced agentic workflows, where the reasoning phase is explicitly decoupled from the action phase to ensure logical consistency [26].

A defining feature of the SEMAA orchestration is its communication protocol; unlike traditional software systems that pass complex, structured data objects between modules, these agents communicate through a unified conversation transcript. In this architecture, cumulative memory is maintained by having every agent write its findings into a shared list of input items, which ensures contextual continuity as subsequent agents "read" the entire history of the session to understand both the final requirements and the reasoning steps that led to them. Furthermore, this linear history functions as a dynamic state management system, granting the PlantUML generator and the Evaluator full visibility into the source documents retrieved by earlier agents throughout the modeling process. The multi-agent design shown in **Figure 5** utilizes three specialist roles to execute the plan created by the Orchestrator:

- The Requirements Agent (Information Retrieval): This agent is designed for grounded reasoning. By utilizing external search tools, it bridges the gap between the user's vague input and technical documentation. It focuses on

extracting engineering specifications and restructuring them into a hierarchical format suitable for modeling.

- The PlantUML Agent (Generative Synthesis): Once the requirements are clarified, this agent focuses exclusively on translation. It converts the synthesized text into formal PlantUML code. By isolating this task, the agent can adhere to strict syntactic protocols without being distracted by the ambiguity of the initial user request.
- The Evaluation Agent (Quality Assurance): The final stage of the design is a dedicated verification loop. The Evaluation Agent acts as a critic, inspecting the generated code for both structural integrity and alignment with the user's intent. This Self-Correction mechanism is vital for maintaining the high precision required in systems engineering, as it can catch and flag errors before they are imported into the MBSE environment.

Through this coordinated workflow, the system transforms a high-level engineering concept into a verified, technical model through a series of governed, collaborative steps.

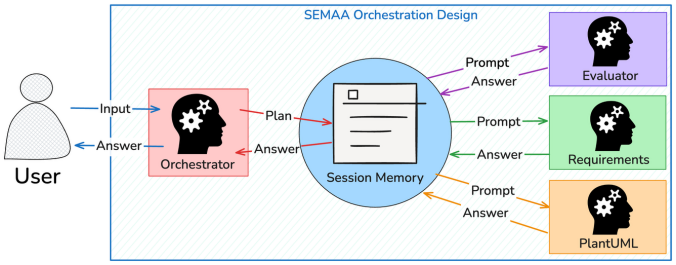

Figure 5: SEMAA Orchestration Agents

5 CASE STUDY AND EVALUATION

5.1 Experiment Design

To evaluate the efficacy of the SEMAA framework and quantify the benefits of multi-agent orchestration in Model-Based Systems Engineering (MBSE), we conducted a series of experiments. The primary motivation for this evaluation was to determine if specialized agentic workflows could surpass the performance and contextual degradation observed in traditional, single-agent systems when applied to complex engineering domains.

Research Questions: In this case study, we seek the answers to the following three research questions:

- RQ1: Does multi-agent orchestration improve the quality of generated diagrams over a single-agent baseline?
- RQ2: How does model choice affect relevance, complexity, correctness, and latency?
- RQ3: What trade-offs exist between output quality and computational efficiency?

Benchmarking: A major challenge in evaluating AI support for systems engineering is the lack of standardized, domain-specific datasets that integrate natural-language requirements with formal outputs. To address this gap, we developed the SEMAA Benchmark, a curated dataset comprising 12 diagram generation tasks specifically targeting the cargo shipbuilding industry. The tasks are grounded in the technical and safety standards of Nippon Kaiji Kyokai (ClassNK), requiring the models to synthesize information from technical rules and documents available to the model. Figure 6 illustrates the design and the process of the case study and evaluation.

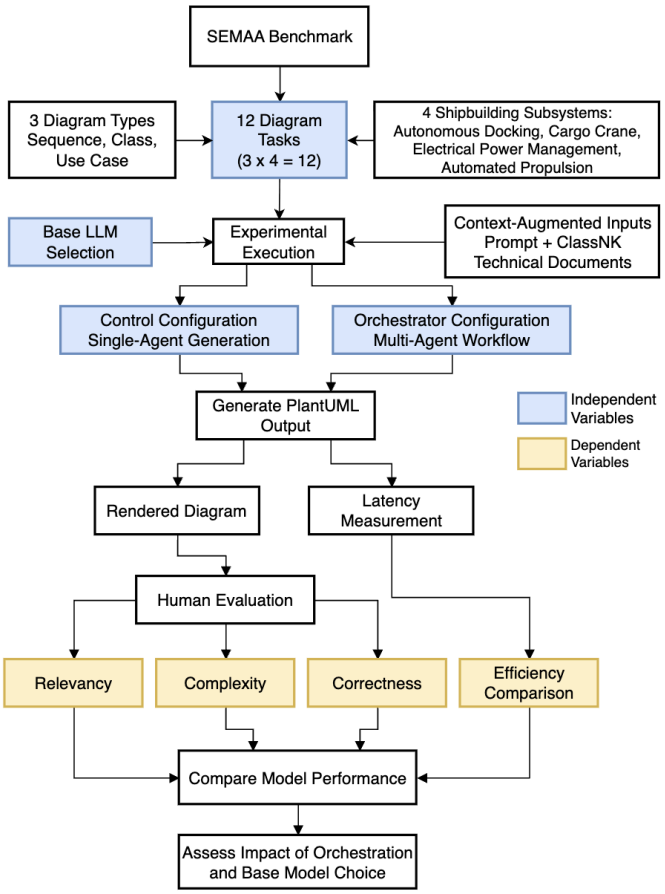

Figure 6: Overview of the SEMAA case study experiment

Dataset generation: As shown in **Figure 6**, the dataset is constructed spanning three Diagram Types (Sequence, Class, and Use Case) across four Mechanical Subsystem domains

Table1: Rubric Evaluation

Criteria\Grade	4	3	2	1
Relevancy	Fully matches the request; all elements included	Mostly matches; minor elements missing	Partially matches; critical elements missing	Does not match; major omissions.
Complexity	Well-structured; appropriate depth and relationships	Moderate detail; clear components shown	Basic structure; limited interactions	Oversimplified; lacks meaningful structure
Correctness	Technically accurate; valid PlantUML syntax.	Mostly accurate; minor formatting issues	Noticeable errors in relationships/logic.	Major syntax/logic errors; fails to render

(Autonomous Docking System, Cargo Crane, Electrical Power Management, and Automated Propulsion). This set of data ensures that agents are tested across varying levels of technical and structural abstraction. Unlike general diagram generation, these prompts are context-augmented with the required documents that the system needs to apply to generate diagrams that are correct with respect to maritime constraints and standards.

Two Configurations for Comparison: The experiment compared two primary configurations:

- A control setup (single-agent configuration), and
- An orchestrator setup (multi-agent configuration).

The control setup was designed as a single-agent baseline. For each task, the single-agent configuration used the same selected backend model, the same user prompt, the same PlantUML output requirement, and the same evaluation rubric as the orchestrated configuration. The key difference was architectural: in the control setup, requirement interpretation, diagram generation, and optional verification were handled within one prompt context, whereas in the orchestrated setup these responsibilities were decomposed across the Orchestrator, Requirements, PlantUML/SysML Generation, and Evaluation agents. This design keeps the task information and model backend constant while isolating the effect of orchestration and role specialization.

As shown in **Figure 6**, in both scenarios, the models were prompted to generate PlantUML code. PlantUML was selected due to its status as a well-established, text-based standard. Given the vast amount of PlantUML documentation and examples available in public training sets, there is a high probability that modern LLMs possess a robust understanding of its syntax, making it an ideal tool for AI-assisted modeling. We tested a wide array of model generations to capture a representative performance spectrum:

- OpenAI Suite: GPT-5.2, GPT-4.1, GPT-5 mini, Codex 5.2, 5 nano, and o4 mini.
- Google Gemini Suite: Gemini 3 Fast and Gemini 3 Thinking

Prompting and Evaluation: Each test involved providing the model with a specific engineering prompt augmented with relevant ClassNK technical excerpts, as shown in **Figure 6**. The resulting PlantUML code and the time taken to generate it (latency) were recorded for each of the 12 diagrams among 14 experimental configurations.

The generated diagrams were then evaluated by two graduate Mechanical Engineering students. These evaluators performed their assessments independently and with no prior knowledge of which specific model configuration produced a given diagram. In addition, prior to assessments, each evaluator completed a comprehensive review of the technical documents that were provided to models to ensure a consistent baseline of domain knowledge before judgment. Each model configuration was tested against all the tasks of the SEMAA Benchmark, and with both evaluators scoring every output using a structured rubric with scores ranging from 1 to 4 across three key criteria, as shown in **Table 1**. The final data points used for our analysis represent the average of scores across all four criteria for every tested configuration.

To demonstrate the capabilities of the SEMAA we present a sample task from SEMAA Benchmark results. The prompt shown in **Table 2** was used to generate the sequence diagram for a ClassNK compliant engine safety system. This task represents a significant challenge for standard generative models because it requires more than just general knowledge of PlantUML syntax and ClassNK compliance regulations. Specifically, it demands logical reasoning in three distinct areas. Firstly, the model must accurately identify the hierarchy of safety actions. For instance, a pressure below 1.0 bar is more critical than 1.5 bar, meaning the "Emergency Shutdown" must have higher priority than "Auto Shutdown" logic. Secondly, the model must assign specific engineering actions to appropriate hardware components e.g., knowing the "Governor" controls the fuel rate while the "ECR" is responsible for logging. Lastly, these systems operate in a continuous loop, thus representing an event within a continuous monitoring requires the model to understand the temporal dependency of events.

The generated diagram, shown in **Figure 7,** demonstrates high technical fidelity. Specifically, the implementation of the blocks correctly captures the conditional safety logic by distinguishing between the critical and cautionary pressure thresholds. Furthermore, the model represents the fuel index reduction by the "Governor" and the flashing alarm on the "Bridge Telegraph", confirming that the domain expert agent successfully extracted these specific requirements from the source documents. The inclusion of a loop block for monitoring the oil pressure illustrates the system's ability to model continuous monitoring tasks rather than one-time execution.

While the generated diagram is technically valid, a domain expert might identify a few missing professional nuances, such as sensor redundancy, specific verification steps, or manual overrides. Despite these potential gaps, SEMAA demonstrates a robust capability for generating valid architectural drafts, effectively serving as a collaborative tool that allows engineers to focus their efforts on high-level design refinements rather than manual syntax construction.

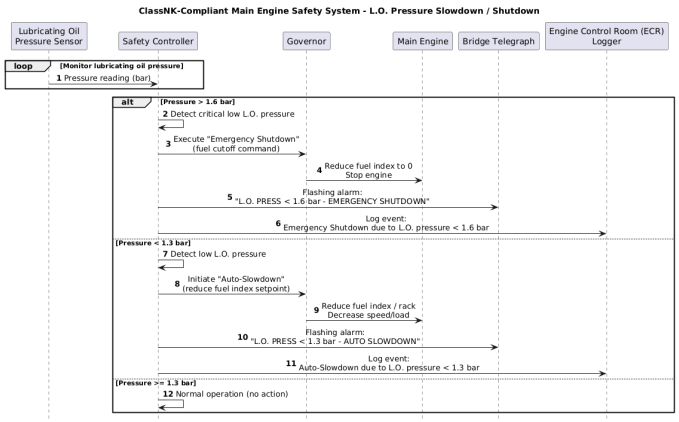

Figure 7: Sample Generated Diagram

Table2: Sample Prompt SEMAA Benchmark

Prompt: *Create a PlantUML sequence diagram for a ClassNK compliant Main Engine Safety System. When the Lubricating Oil Pressure Sensor drops below 1.5 bar, the Safety Controller must initiate an 'Auto-Slowdown.' The sequence should show the Governor reducing fuel index, the Bridge Telegraph receiving a flashing alarm, and the Engine Control Room (ECR) logging the event. Include an 'alt' block where, if pressure drops below 1.0 bar, the system executes an 'Emergency Shutdown'*

5.2 RESULTS AND DISCUSSION

The results of the model performance benchmark, as illustrated in **Figure 8**, demonstrate a clear performance advantage for Orchestrator configurations over their Control baselines across all qualitative metrics. The Orchestrator 5.2 configuration emerged as the top-performing model, achieving the highest mean scores in all criteria. A consistent trend is also observed where the integration of the orchestrator architecture enhances the model's capability to generate diagrams with more technical accuracy compared to a single-agent setup. On the other hand, smaller models, such as the GPT 5 nano and GPT o4 mini variants, exhibited a decline in performance, particularly in Correctness and Complexity.

Computational efficiency, measured via average latency, showed significant fluctuations across the tested configurations, often independent of qualitative output quality. Interestingly, the Orchestrator 5.2 model maintained a relatively low latency of approximately 10 s, which was notably lower than its Control 5.2 counterpart (~14 s). The most substantial latency penalty was observed in the Orchestrator 5 nano configuration, which peaked at nearly 40s, representing the least efficient performance in the dataset despite its lower qualitative scores. In contrast, the Control 5.2 Codex configuration offered the lowest latency at roughly 4s, though this speed came at the cost of a marked reduction in Correctness and Complexity. These results suggest that while the 5.2 tier provides the most robust modeling capabilities, the Orchestrator architecture optimizes these models to perform more efficiently than standard single-agent configurations

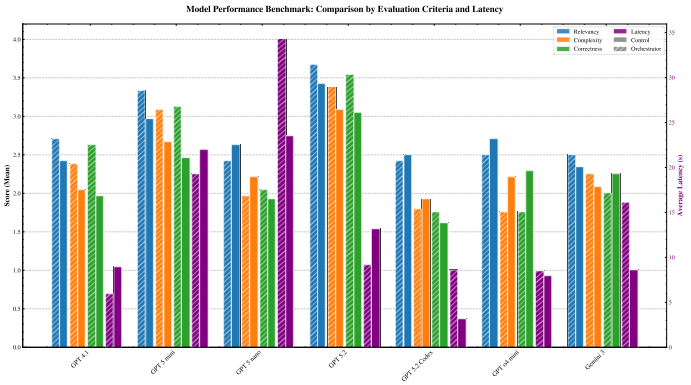

Figure 8: Models Performance Results

5.2.1 Impact of Orchestration on Model Performance

A primary finding of this study is the performance gain achieved by the Orchestrator configuration compared to the Control setup. While both configurations utilize the same underlying engine, the Orchestrator configuration demonstrated better consistency, particularly in Relevancy and Correctness. The superior performance of the orchestrated configuration is likely related to the separation of expert models within the multi-agent architecture. In traditional single-agent systems, a single model must simultaneously process unstructured engineering documentation while adhering to SysML's strict syntactic rules, which often degrades output quality. By assigning high-level task decomposition to a specialized orchestrator and isolating the modeling agent from document context, the framework effectively mitigates the lost in the middle phenomenon, where retrieval accuracy significantly decreases as relevant information is buried within long input sequences

5.2.2 Impact of Different Base Models

The data reveals a clear gap between different generations of models. High-tier models, specifically the GPT 5.2 and GPT 5 mini, consistently achieved high Complexity scores, indicating the ability to generate SysML structures with nested blocks and multiple relationship types. In contrast, smaller or older models, such as GPT 4.1 and GPT 5 nano, tended to produce oversimplified diagrams, often failing to capture the required hierarchies. The most striking disparity was observed in Correctness. Specially tuned models, such as the Codex, frequently produced invalid code. This suggests that PlantUML syntax is a highly specialized domain where general-purpose coding generation models fail to maintain syntactic correctness. The orchestrator configuration successfully bridged this gap by employing a specialized "PlantUML Expert" agent that focuses exclusively on syntax verification, ensuring that even complex designs can be produced without error.

5.2.3 Latency-Performance Trade-off

An analysis of the average latency reveals a surprising result in the multi-agent system. Conventionally, one might expect that a multi-agent workflow involving inter-agent communication and task delegation would increase latency considerably. However, our data shows that Orchestrator GPT 5.2 achieved an average latency of approximately 9–10 seconds, notably faster than GPT Control 5.2, which frequently peaked at over 20 seconds. We hypothesize that the single-agent control model suffers from reasoning bloat, in which the model spends excessive computational time reconciling a massive context window that simultaneously contains raw documents and modeling rules. By contrast, the Orchestrator breaks the problem into smaller, more focused prompts, allowing the final modeling agent to generate code more decisively and rapidly. While the Orchestrator GPT 5 mini showed high relevance, its latency was significantly higher and more volatile (peaking at 50 seconds), suggesting that even smaller models that require less computational power might fall into reasoning bloat.

The results underscore a critical lesson for the development of agentic AI in MBSE: raw intelligence is insufficient without

structured orchestration. The failure of the GPT nano and GPT o4 mini models, despite being part of the same suite, indicates that the architectural complexity of system engineering exceeds the reasoning threshold that smaller models have not yet reached. However, for high-capacity models, the shift from a monolithic single-agent system to a collaborative team of specialists provides a dual benefit: it increases the architectural integrity of the system design while simultaneously reducing the time an engineer must wait for a response. For professional engineering applications, the Orchestrator GPT 5.2 configuration offers the optimal balance, providing high-fidelity, syntactically correct, and low-latency assistance for real-time engineering collaboration.

5.3 Limitations

This case study has several limitations that should be considered when interpreting the results. First, the benchmark is small, consisting of only 12 tasks, and it covers only three diagram types: sequence, class, and use case diagrams. While these tasks provide an initial basis for comparison, they do not capture the full diversity of MBSE artifacts and system modeling situations.

Second, the evaluation is limited to the cargo shipbuilding domain using ClassNK-based technical context. This makes the benchmark realistic and domain-grounded, but it also limits the generalizability of the findings to other engineering domains.

Third, the output quality was assessed through a human-scored rubric based on relevancy, complexity, and correctness. Although practical, this introduces subjectivity, especially because the evaluators were Mechanical Engineering students rather than professional systems engineers or MBSE experts.

Finally, the framework depends on PlantUML as an intermediate representation, and the current evaluation does not include full formal semantic validation beyond syntax, renderability, and rubric-based correctness. As a result, valid-looking diagrams may still fall short of stricter model-level semantic requirements.

Despite these limitations, the case study provides useful initial evidence that multi-agent orchestration can improve AI-assisted engineering diagram generation under the tested conditions.

6 CONCLUSION

This research demonstrates that developing a robust agentic system for complex engineering tasks, particularly within the domain of Model-Based Systems Engineering (MBSE), is entirely feasible. We successfully developed a proof-of-concept framework that bridges the gap between probabilistic AI reasoning and deterministic architectural modeling. By integrating a professional MBSE modeler with a custom plugin and a bi-directional interpreter, the system effectively uses PlantUML as an intermediate representation language to generate and synchronize formal diagrams from natural-language prompts.

The primary contributions and novelty of this work lie in the implementation and evaluation of a multi-agent orchestration strategy for modeling tasks found in systems engineering. More specifically, the contributions include:

- **SEMAA framework**: The paper proposes SEMAA, a proof-of-concept agentic framework for supporting MBSE using conversational AI.
- **Tool-integrated implementation**: The framework is realized as a Visual Paradigm plugin and uses PlantUML as an intermediate representation to connect natural-language interaction with formal diagram generation.
- **Multi-agent orchestration for engineering modeling**: The paper introduces a specialized orchestration strategy in which different agents handle requirement interpretation, diagram synthesis, and evaluation, instead of relying on a single-agent workflow.
- **Case-study-based evaluation**: The paper develops a domain-specific benchmark for shipbuilding and shows that the orchestrated SEMAA configuration outperforms single-agent baselines in terms of relevance, complexity, and correctness.

Our findings indicate that a specialized team of agents outperforms monolithic, single-agent configurations across critical engineering metrics, including diagram relevancy, complexity, and technical correctness. Notably, the multi-agent approach achieves these qualitative improvements while maintaining - and in some cases reducing - computational latency compared to standard single-agent models. This suggests that structured orchestration is a key factor in mitigating the context rot and reasoning overhead typically found in large-scale engineering design sessions.

Looking forward, several avenues for future research remain. We intend to expand the evaluation dataset to encompass a broader range of engineering domains to further validate the system's cross-disciplinary capabilities. Additionally, we aim to refine the internal logic of the multi-agent orchestration to support even more complex design verification loops. Finally, a significant future goal is the development of a centralized, tool-agnostic user interface leveraging the Model Context Protocol (MCP). This evolution would allow the SEMAA framework to connect seamlessly to multiple engineering tools, creating a truly integrated and collaborative AI-human modeling environment

ACKNOWLEDGEMENTS

This paper is based on the work supported in part by the industrial sponsors: BEMAC Corporation, ClassNK, and MTI Co., Ltd. The authors are grateful for their support and collaboration on this research. The authors acknowledge the Center for Advanced Research Computing (CARC) at the University of Southern California for providing computing resources that have contributed to the research results reported within this publication. URL: https://carc.usc.edu.

REFERENCES

[1] PlantUML, “PlantUML.” Available: https://plantuml.com/.

[2] Henderson, K., and Salado, A., 2021, "Value and Benefits of Model-based Systems Engineering (MBSE): Evidence from the Literature," Syst. Eng., **24**(1), pp. 51–66. https://doi.org/10.1002/sys.21566.

[3] Hause, M. and others, 2006, "The SysML Modelling Language," *15th European Systems Engineering Conference*, pp. 1–12.

[4] Dassault Systèmes, 2024, "No Magic Cameo Systems Modeler." [Online]. Available: https://www.3ds.com.

[5] Visual Paradigm, 2026, "Visual Paradigm User's Guide." [Online]. Available: https://www.visual-paradigm.com/.

[6] Foundation, E., 2026, "SysON - The NextGen SysML Modeling Tool." [Online]. Available: https://mbse-syson.org/.

[7] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T., 2023, "Toolformer: Language Models Can Teach Themselves to Use Tools." https://doi.org/10.48550/arXiv.2302.04761.

[8] Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O., 2024, "SWE-Agent: Agent-Computer Interfaces Enable Automated Software Engineering." https://doi.org/10.48550/arXiv.2405.15793.

[9] He, Q., Nan, J., Jiao, J., Tang, L., Xu, X., Sun, M., Wang, Q., and Yan, M., 2025, "Z-Space: A Multi-Agent Tool Orchestration Framework for Enterprise-Grade LLM Automation." https://doi.org/10.48550/arXiv.2511.19483.

[10] Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., and Wang, C., 2023, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation." https://doi.org/10.48550/arXiv.2308.08155.

[11] Gao, D., Li, Z., Pan, X., Kuang, W., Ma, Z., Qian, B., Wei, F., Zhang, W., Xie, Y., Chen, D., Yao, L., Peng, H., Zhang, Z., Zhu, L., Cheng, C., Shi, H., Li, Y., Ding, B., and Zhou, J., 2024, "AgentScope: A Flexible yet Robust Multi-Agent Platform." https://doi.org/10.48550/arXiv.2402.14034.

[12] Fourney, A., Bansal, G., Mozannar, H., Tan, C., Salinas, E., Erkang, Zhu, Niedtner, F., Proebsting, G., Bassman, G., Gerrits, J., Alber, J., Chang, P., Loynd, R., West, R., Dibia, V., Awadallah, A., Kamar, E., Hosn, R., and Amershi, S., 2024, "Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks." https://doi.org/10.48550/arXiv.2411.04468.

[13] Islam, M. A., Ali, M. E., and Parvez, M. R., 2024, "MapCoder: Multi-Agent Code Generation for Competitive Problem Solving." https://doi.org/10.48550/arXiv.2405.11403.

[14] Huang, D., Zhang, J. M., Luck, M., Bu, Q., Qing, Y., and Cui, H., 2024, "AgentCoder: Multi-Agent-Based Code Generation with Iterative Testing and Optimisation." https://doi.org/10.48550/arXiv.2312.13010.

[15] Ishibashi, Y., and Nishimura, Y., 2024, "Self-Organized Agents: A LLM Multi-Agent Framework toward Ultra Large-Scale Code Generation and Optimization." https://doi.org/10.48550/arXiv.2404.02183.

[16] Zhu, Y., Liu, C., He, X., Ren, X., Liu, Z., Pan, R., and Zhang, H., 2025, "AdaCoder: An Adaptive Planning and Multi-Agent Framework for Function-Level Code Generation." https://doi.org/10.48550/arXiv.2504.04220.

[17] Pan, R., Zhang, H., and Liu, C., 2025, "CodeCoR: An LLM-Based Self-Reflective Multi-Agent Framework for Code Generation." https://doi.org/10.48550/arXiv.2501.07811.

[18] Hasan, M. H., Dihan, M. L., Hashem, T., Ali, M. E., and Parvez, M. R., 2025, "MapAgent: A Hierarchical Agent for Geospatial Reasoning with Dynamic Map Tool Integration." https://doi.org/10.48550/arXiv.2509.05933.

[19] Sivakumaran, N., Chen, J. C.-Y., Wan, D., Zhang, Y., Yoon, J., Stengel-Eskin, E., and Bansal, M., 2025, "DART: Leveraging Multi-Agent Disagreement for Tool Recruitment in Multimodal Reasoning." https://doi.org/10.48550/arXiv.2512.07132.

[20] Li, J., Huo, N., Gao, Y., Shi, J., Zhao, Y., Qu, G., Wu, Y., Ma, C., Lou, J.-G., and Cheng, R., 2024, "Tapilot-Crossing: Benchmarking and Evolving LLMs Towards Interactive Data Analysis Agents." https://doi.org/10.48550/arXiv.2403.05307.

[21] Trirat, P., Jeong, W., and Hwang, S. J., 2025, "AutoML-Agent: A Multi-Agent LLM Framework for Full-Pipeline AutoML." https://doi.org/10.48550/arXiv.2410.02958.

[22] Sun, M., Han, R., Jiang, B., Qi, H., Sun, D., Yuan, Y., and Huang, J., 2025, "LAMBDA: A Large Model Based Data Agent." https://doi.org/10.48550/arXiv.2407.17535.

[23] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D., 2023, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." https://doi.org/10.48550/arXiv.2201.11903.

[24] Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P., 2023, "Lost in the Middle: How Language Models Use Long Contexts." https://doi.org/10.48550/arXiv.2307.03172.

[25] Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., Wang, Z., Yau, S. K. S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., and Schmidhuber, J., 2024, "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework." https://doi.org/10.48550/arXiv.2308.00352.

[26] Wang, L., Xu, W., Lan, Y., Hu, Z., Lan, Y., Lee, R. K.-W., and Lim, E.-P., 2023, "Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models." https://doi.org/10.48550/arXiv.2305.04091.

7 APPENDIX

7.1 Appendix A: Agent Architecture Specification

This appendix summarizes the implementation-level agent specification used in the SEMAA orchestration workflow. The four agents share the same selectable model backend; the

backend can be chosen from the available OpenAI or Gemini model used in the experiment. The distinction among agents is therefore not defined by using different vendors, but by assigning each role a different instruction context, input scope, output format, and tool access. In the implementation, the agents are executed as separate role-specific calls or prompt contexts coordinated through the shared session memory.

Orchestrator Agent: The Orchestrator Agent receives the original user request, the current session memory, the available agent role descriptions, and the current modeling context when available. Its responsibility is to interpret the user's intent, decompose the task into ordered subtasks, and decide which specialist agents should run and in what sequence. Its output is an execution plan and a set of comprehensive role-specific prompts written to the session memory. A representative instruction prompt is: analyze the user request, identify the required modeling objective, determine the necessary information-retrieval, generation, and evaluation steps, and produce clear instructions for each specialist agent. The Orchestrator does not directly create the final diagram; instead, it manages task planning, role assignment, and information flow.

Requirements Agent: The Requirements Agent receives the Orchestrator's prompt, the user request, relevant session memory, and any available project or domain context. Its responsibility is to extract, organize, and clarify the engineering requirements needed for diagram generation. When supporting documents are available, this agent may use simple file reading or retrieval tools to inspect the relevant text and summarize the constraints. Its output is a structured requirements summary, including actors, components, relationships, constraints, assumptions, and any unresolved ambiguities. A representative instruction prompt is: read the assigned prompt and available context, identify the engineering requirements relevant to the requested SysML diagram, organize them into a modeling ready structure, and write the result back to session memory.

PlantUML/SysML Generation Agent: The PlantUML/SysML Generation Agent receives the Orchestrator's prompt, the structured requirements generated by the Requirements Agent, the current diagram context, and any constraints stored in session memory. Its responsibility is to translate the modeling intent into PlantUML based SysML diagram definitions. Its output is PlantUML code and, when needed, a brief natural language rationale explaining the major modeling choices. This agent may use file writing tools or plugin output channels to pass the generated representation to the interpreter. A representative instruction prompt is: convert the structured requirements into valid PlantUML representing the requested SysML diagram type, preserve the user's intent, use clear element names and relationships, and return only the diagram code plus a concise rationale.

Evaluation Agent: The Evaluation Agent receives the generated PlantUML/SysML representation, the original user request, the structured requirements summary, and the relevant session memory. Its responsibility is to verify whether the generated diagram is syntactically valid, structurally coherent, and aligned with the user's modeling intent. Its tool access includes simple file-reading and file-writing operations and, when available, a syntax-checking or rendering check for the PlantUML output. Its output is an evaluation report identifying pass/fail status, syntax errors, missing elements, inconsistent relationships, and recommended corrections. A representative instruction prompt is: compare the generated model against the original request and extracted requirements, run or request a syntax check when possible, identify errors or omissions, and return either an approval or specific revisions for the generation agent.

The shared session memory acts as the communication medium among agents. Each agent reads the prompt and context assigned by the Orchestrator, performs its role-specific task, and writes its result back to the session memory. This design allows the workflow to preserve traceability while avoiding the need to pass the full project history to every agent. It also allows the architecture to be deployed flexibly: the same roles can be implemented as separate LLM calls, specialized prompts over the same backend model, or tool-enabled modules within a larger MBSE plugin architecture.